БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики и информатики

Кафедра технологий программирования

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

Мониторинг в распределённых системах

Дисциплина «Непрерывное интегрирование и сборка программного обеспечения»

Задание 1

Выполнил: Шагелдиев
Касым
Студент 4 курса, группа 13

Минск, 2026

1. Цель и постановка задачи

Цель работы: изучить сбор телеметрии Python-приложения, подключить трассировку OpenTelemetry, настроить просмотр запросов в Jaeger и запуск приложения в Docker Compose.

Для группы 13 согласно критериям методических указаний (страница 6) выполняется задание 1: создание Flaskприложения, установка OpenTelemetry, подключение трассировки, развёртывание Jaeger и упаковка приложения в Docker. Исходные файлы размещаются в каталоге task1.

Среда выполнения

Windows, PowerShell и Docker Desktop с Linux-контейнерами через WSL 2. По выводу команд: Git 2.56.0.windows.1, Python 3.14.7, Docker 29.8.1, Docker Compose v5.5.1, WSL 3.0.3.0. Версия OpenTelemetry SDK в трассировках: 1.45.2. Образ Jaeger: 2.22.0; базовый образ приложения: python:3.14-slim.

Схема работы

Клиент выполняет GET /rolldice. FlaskInstrumentor создаёт серверный span. Внутри обработчика функция roll создаёт дочерний span, записывает число в атрибут dice.result и добавляет событие dice_rolled. BatchSpanProcessor передаёт spans через OTLP/HTTP в Jaeger.
Пользователь просматривает результат через веб-интерфейс.

Из Windows приложение доступно по адресу http://127.0.0.1:5000/rolldice, Jaeger — http://localhost:16686. В сети Compose приложение отправляет данные на http://jaeger:4318/v1/traces. Имя jaeger разрешается средствами сети Compose [4].

Этапы реализации

Сначала проверены Docker и запуск hello-world. Создано приложение с генератором чисел 1–6. Затем настроен консольный экспорт spans и проверена связь trace_id/parent_id. После запуска Jaeger консольный экспорт заменён на OTLP/HTTP. На последнем этапе приложение перенесено в контейнер и выполнены HTTP-запросы к стенду. Репозиторий: **https://github.com/KasymShageldiyev/lab3nispo**

2. Код приложения

Файл task1/app.py. Листинг соответствует итоговому коду, использованному в ходе выполнения; различия форматирования не меняют поведения.

```
import os from random
import randint
from flask import Flask from opentelemetry import trace from
opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry.instrumentation.flask import FlaskInstrumentor from
opentelemetry.sdk.resources import Resource from opentelemetry.sdk.trace
import TracerProvider from opentelemetry.sdk.trace.export import
BatchSpanProcessor
resource = Resource.create({"service.name": "lab3-
flask"}) provider = TracerProvider(resource=resource)
exporter = OTLPSpanExporter(     endpoint=os.getenv(
"OTEL_EXPORTER_OTLP_TRACES_ENDPOINT",
        "http://localhost:4318/v1/traces",
    ) )
provider.add_span_processor(BatchSpanProcessor(expor
ter) ) trace.set_tracer_provider(provider) tracer =
trace.get_tracer(__name__) app = Flask(__name__)
FlaskInstrumentor().instrument_app(app)


def roll():     with
```

```
tracer.start_as_current_span("roll") as span:
result = randint(1, 6)
span.set_attribute("dice.result", result)
span.add_event("dice_rolled")         return
result


@app.route("/rolldice")
def roll_dice():
    return str(roll())
```

3. Конфигурация контейнеров

task1/Dockerfile

```
FROM python:3.14-slim
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
WORKDIR /app COPY
requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "-m", "flask", "--app", "app", "run", "--host=0.0.0.0", "--
```

```
port=5000"] task1/compose.yaml services:   jaeger:     image:

cr.jaegertracing.io/jaegertracing/jaeger:2.22.0
ports:       - "127.0.0.1:16686:16686"       -

"127.0.0.1:4318:4318"   app:     build:      context:

.     ports:       - "127.0.0.1:5000:5000"

environment:       OTEL_EXPORTER_OTLP_TRACES_ENDPOINT:
http://jaeger:4318/v1/traces     depends_on:       - jaeger
```

Файлы исключений

В .gitignore и task1/.dockerignore исключены .venv/, __pycache__/, *.py[cod] и .env. В .dockerignore дополнительно исключён .git/.

4. Зависимости и запуск

Основные пакеты: Flask, opentelemetry-api, opentelemetry-sdk, opentelemetry-instrumentation-flask и opentelemetry-exporter-otlp-proto-http. Полный список установленных версий фиксируется командой pip freeze в task1/requirements.txt. Источником точного списка является этот файл репозитория; полный вывод pip freeze в материалы отчёта не передавался. **Запуск из корня репозитория**

```
docker compose -f task1/compose.yaml up -d --build

docker compose -f task1/compose.yaml ps
docker compose -f task1/compose.yaml exec app python -m pip check
```

Пять проверочных запросов в PowerShell

```
1..5 | ForEach-Object {     Invoke-RestMethod -Uri

"http://127.0.0.1:5000/rolldice"
}
```

После выполнения запросов необходимо подождать отправки пакета spans, открыть Jaeger, выбрать сервис lab3flask и нажать Find Traces. В трассировке раскрыть GET /rolldice и roll.

Журнал и остановка

```
docker compose -f task1/compose.yaml logs --tail 80 app jaeger

docker compose -f task1/compose.yaml down
```

Ограничения учебного стенда

Используется сервер разработки Flask. Jaeger хранит трассировки в памяти, поэтому они не сохраняются после перезапуска процесса. depends_on определяет порядок запуска, но сам по себе не подтверждает готовность Jaeger принимать данные. Сбор метрик и централизованных логов не настраивался: это задания 2 и 3.

Пересоздание стенда с нуля и нагрузочное тестирование в предоставленных результатах не зафиксированы. Приведённые ниже проверки подтверждают успешную работу текущего запуска.

5. Проверка контейнеров и HTTP

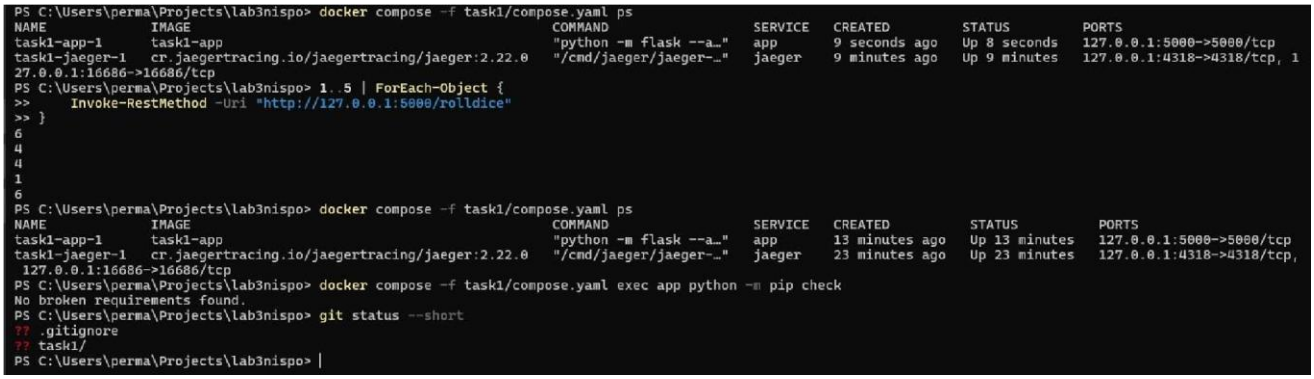


Рисунок 1. Сервисы app и jaeger имеют статус Up. Проверка зависимостей внутри app: No broken requirements found. В выводе Compose опубликованы порты 5000 для приложения, 16686 для интерфейса Jaeger и 4318 для OTLP/HTTP. Проверка pip check не обнаружила несовместимых требований установленных пакетов.

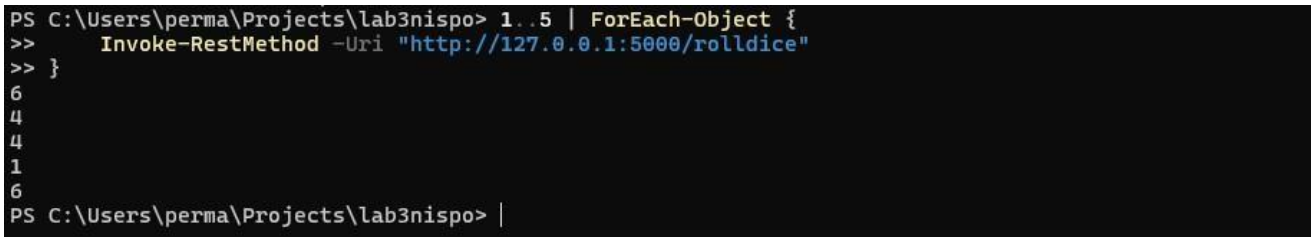


Рисунок 2. Пять последовательных HTTP-запросов вернули 6, 4, 4, 1, 6.

Все пять ответов находятся в допустимом диапазоне 1–6. Повторение значения допустимо. Эта проверка подтверждает обработку запросов и диапазон наблюдаемых ответов, но не проверяет статистическую равномерность генератора.

6. Проверка трассировки в Jaeger

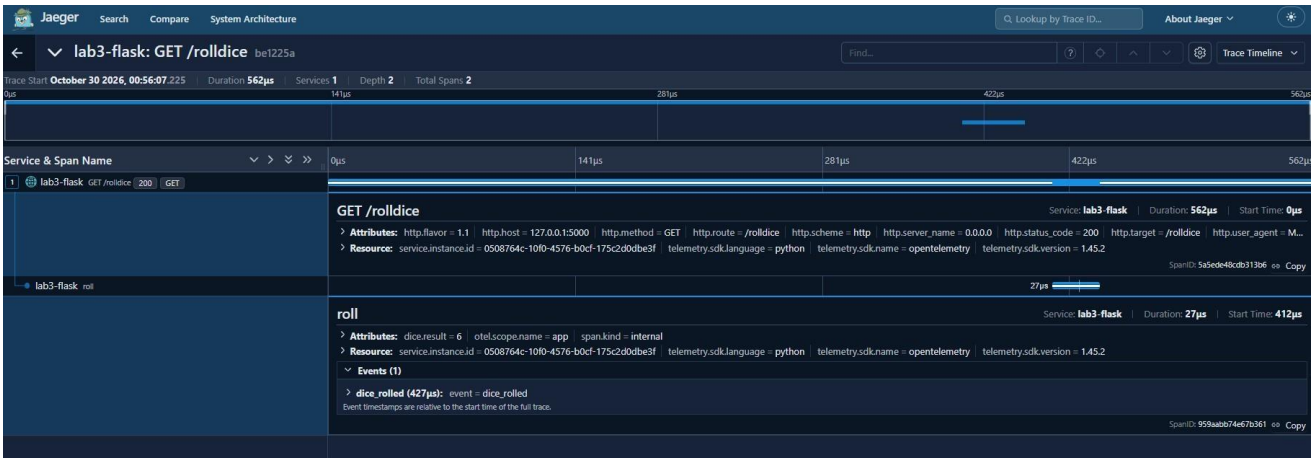


Рисунок 3. Трассировка контейнеризированного приложения: GET /rolldice и дочерний span roll.

На снимке: начало запроса 30.10.2026 в 00:56:07.225, длительность 562 мкс, один сервис, два span, глубина 2. HTTP-статус — 200. Span roll длится 27 мкс, содержит dice.result = 6 и событие dice_rolled. Имя области инструментирования — app.

В ранее проверенном консольном выводе оба span имели trace_id 0x44ea442d89fbc5632a0153083ed9adfe. Значение parent_id дочернего span совпадало со span_id серверного span: 0x86063810d7b8fc31. Это подтверждает корректную родительскую связь.

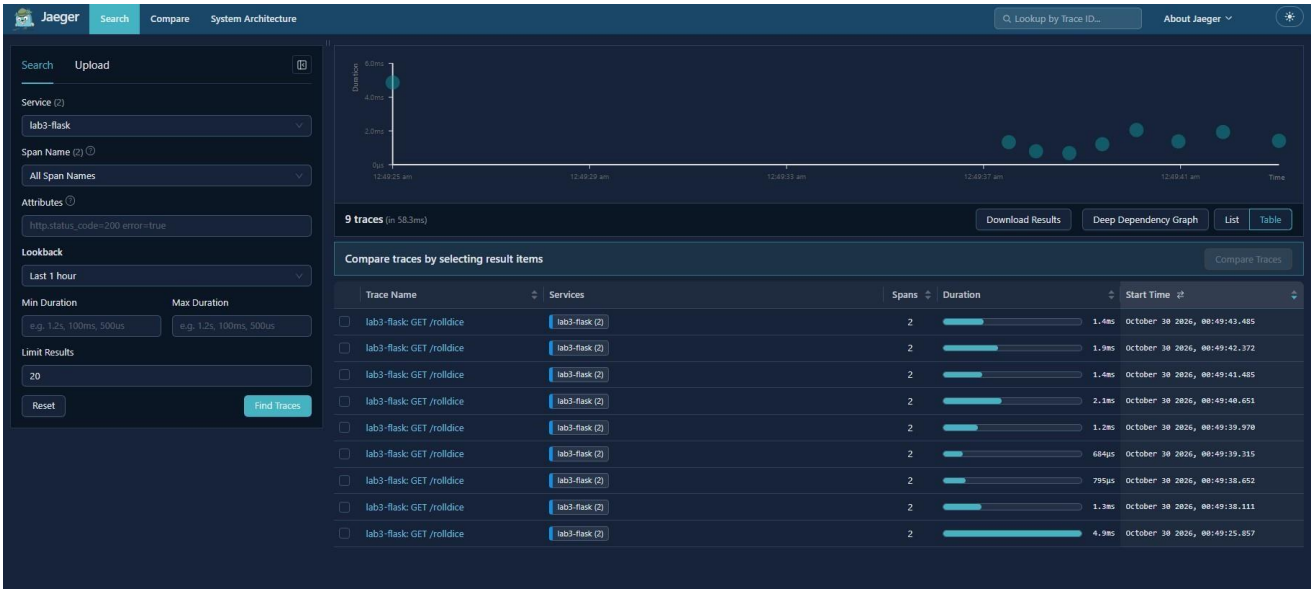


Рисунок 4. Поиск трассировок на предыдущем этапе локального запуска Flask: 9 запросов, по два span в каждом. Этот снимок относится к этапу до контейнеризации приложения.

7. Контрольные вопросы: 1–5

1. Что такое OpenTelemetry? Атрибуты, события, контекст, журналы,трассировки, показатели?

OpenTelemetry — набор открытых API, SDK и инструментов для получения и передачи телеметрии. Атрибут — пара ключ–значение; событие span — запись с временем; контекст переносит сведения о текущей операции. Логи описывают события, трассировки — выполнение запросов, метрики — численные измерения. В работе dice.result — атрибут, dice_rolled — событие [1, 2].

2. Что такое наблюдаемость? Надёжность и показатели?

Наблюдаемость позволяет судить о внутреннем состоянии системы по её внешним сигналам. Надёжность — способность выполнять требуемые функции в заданных условиях.
SLI измеряет качество, например долю успешных запросов; SLO задаёт целевое значение SLI. Для анализа используют доступность, задержку, частоту ошибок и загрузку ресурсов.

3. Что такое трассировка и распределённая трассировка?

Трассировка состоит из связанных spans. Распределённая трассировка объединяет операции нескольких сервисов через общий trace_id и передачу контекста, например в заголовке traceparent. SDK создают spans, экспортёр отправляет их в систему анализа. Проблемы: потеря контекста, асинхронные вызовы, повторные запросы, рассинхронизация часов, объём данных и sampling. В данном стенде инструментирован один сервис.

4. Сигналы OpenTelemetry: трассировки, метрики, логи?

Трассировки показывают путь и длительность запроса; метрики описывают агрегированное состояние во времени; логи содержат записи об отдельных событиях. Связь логов с trace_id помогает перейти от сообщения к конкретному запросу. В работе экспортируются только трассировки.

5. Инструменты OpenTelemetry: автоматические, ручные, библиотеки?

Автоматическое инструментирование добавляет телеметрию через агенты и библиотеки интеграции. Ручное инструментирование использует API в коде приложения. Здесь FlaskInstrumentor создаёт HTTP-spans, а start_as_current_span создаёт собственный span roll. SDK управляет обработкой и экспортом [2].

7. Контрольные вопросы: 6–11

6. Компоненты OpenTelemetry?

Спецификация определяет согласованные модели и API. API используется приложением, SDK реализует сбор и обработку. Библиотеки инструментирования интегрируются с фреймворками. Экспортёры передают данные. Collector принимает, обрабатывает и экспортирует телеметрию; в этой работе отдельный Collector не запускается.

7. Что такое ресурс OpenTelemetry?

Resource описывает источник телеметрии: сервис, экземпляр, среду и другие признаки. Его атрибуты относятся к источнику в целом. В работе service.name равен lab3-flask; dice.result относится к конкретной операции и потому записан в span.

8. Что такое экспортёры OpenTelemetry?

Экспортёр передаёт телеметрию получателю. ConsoleSpanExporter использовался для отладки; OTLPSpanExporter — для отправки traces в Jaeger. OTLP поддерживает
HTTP/protobuf и gRPC. Процессор spans определяет момент и способ передачи завершённых spans экспортёру [3].

9. Что такое Prometheus и как он работает?

Prometheus — система мониторинга временных рядов. Обычно периодически опрашивает
HTTP-endpoints метрик по pull-модели, хранит значения с метками, выполняет PromQL-запросы и правила. Alertmanager обрабатывает уведомления о сработавших оповещениях.

10. Основные концепции Prometheus?

Временной ряд определяется именем метрики и набором labels. Sample содержит значение и время. Instance — опрашиваемая цель, job — группа целей. Scrape — получение метрик. PromQL используется для выборки и вычислений; exporters предоставляют метрики внешних систем.

11. Типы метрик?

Counter увеличивается и может сбрасываться при перезапуске. Gauge может расти и уменьшаться. Histogram описывает распределение наблюдений, их число и сумму; классический вариант использует buckets. Summary вычисляет квантили на стороне клиента, а также число и сумму наблюдений. Квантили summary нельзя напрямую агрегировать между экземплярами [6].

7. Контрольные вопросы: 12–16

12. Запрос и преобразование данных Grafana?

Панель отправляет запрос в выбранный источник на его языке, например PromQL. Преобразования результата позволяют фильтровать поля, объединять таблицы, группировать строки и вычислять значения перед визуализацией. Последовательность преобразований влияет на итог.

13. Источники данных Grafana?

Источник данных — настроенное подключение через плагин к хранилищу или API. Примеры: Prometheus для метрик, Loki для логов, Jaeger для трассировок, PostgreSQL для табличных данных. В настройках указываются адрес, аутентификация и параметры запросов.

14. Панели мониторинга Grafana?

Dashboard объединяет панели визуализации: графики, таблицы, статистику и логи. Панель имеет запросы и настройки отображения. Диапазон времени, переменные и фильтры позволяют исследовать данные; конфигурацию dashboard можно сохранять и переносить
[7].

15. Процесс обработки журналов Grafana Loki?

Агент читает записи, добавляет метки и передаёт их в Loki по push-модели. Loki объединяет записи в потоки по меткам, сохраняет сжатые блоки и индексирует метки, а не полное содержимое строк. LogQL выбирает потоки и фильтрует записи; Grafana отображает результат [8].

16. Варианты сбора журналов для Grafana Loki?

Возможны агент на узле, сбор stdout/stderr контейнеров, чтение файлов, приём от приложения или через OpenTelemetry Collector. Примеры средств — Grafana Alloy и интеграции Fluent Bit. В методичке также описан Promtail. Выбор зависит от источника логов, формата и способа развёртывания; в этой работе сбор логов не реализован.

8. Выводы и источники

Создано Flask-приложение с маршрутом /rolldice. Настроены серверный и дочерний spans, атрибут результата и событие броска. Проверена отправка OTLP/HTTP в Jaeger. Приложение упаковано в Docker и запущено совместно с Jaeger через Compose. Работоспособность подтверждена ответами HTTP, двумя активными контейнерами, проверкой зависимостей и трассировками в интерфейсе Jaeger.

Материалы подготовлены для публикации в каталоге task1, отчёт — в docs/report.pdf. Публикация в GitHub выполняется отдельным шагом и не подтверждается приведёнными скриншотами.

Методические указания

НИиСПО Лабработа3 2026-2027.pdf, БГУ, 2026; страницы 6–10.

1. **OpenTelemetry Concepts** https://opentelemetry.io/docs/concepts/
2. **Python Instrumentation**

https://opentelemetry.io/docs/languages/python/instrumentation/

3. **Python Exporters**

https://opentelemetry.io/docs/languages/python/exporters/

4. **Docker Compose Networking** https://docs.docker.com/compose/how-tos/networking/
5. **Jaeger Getting Started** https://www.jaegertracing.io/docs/2.22/getting-started/

6. Prometheus Metric Types

https://prometheus.io/docs/concepts/metric_types/

7. Grafana Fundamentals

https://grafana.com/docs/grafana/latest/fundamentals/

8. Loki Overview https://grafana.com/docs/loki/latest/get-started/overview/